Welcome to

CCS203

Data Structures and Algorithms

Assessment

Assessment method	Week No	Weight (Marks)
Assignments and projects	Weekly	15
Quizzes	5 and 13	10
Mid-Term examinations	8	15
Oral	14	10
Practical exam	14	10
Final exam	End of semester	40

Introductions



Name



Preferred Name

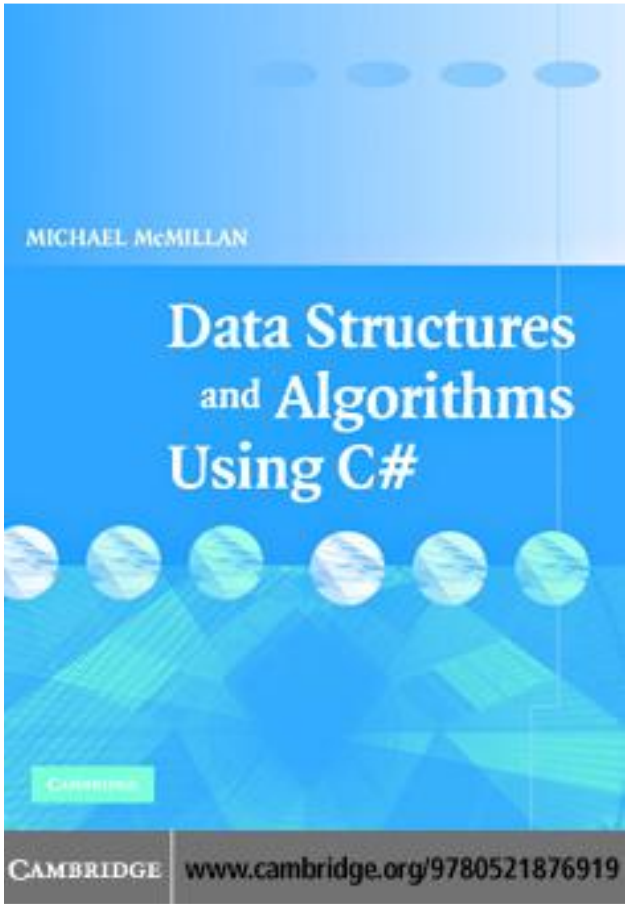


Interesting Fact / Hobby / Excited to Learn

Textbook

Michael Mcmillan,

Data Structures And Algorithms Using C#



What Is Data Modeling?

A real-world application is reduced, top-down, to a small set of well-understood problems — and its data is described abstractly in terms of fundamental structures.

Data Structure

The organized collection of data within a program — how it's arranged and stored.

Algorithm

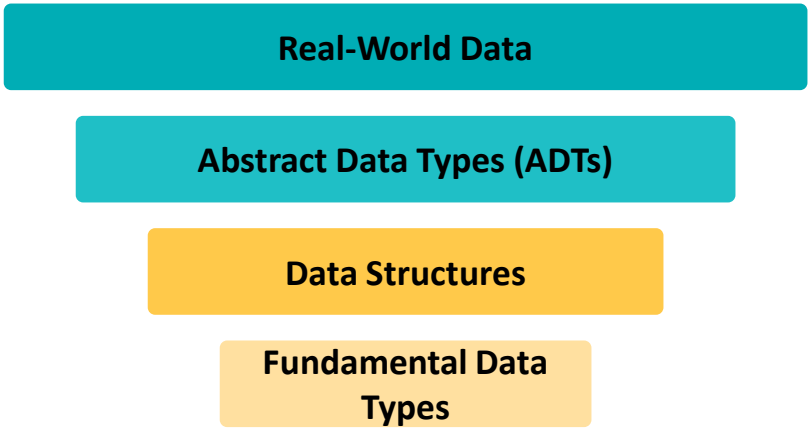
A precise procedure — a well-defined sequence of operations — for solving a specific, well-specified problem.

Data Structures + Algorithms = Real-World Application

Data and the operations on it are two faces of the same object — neither can be designed or understood independently of the other.

The Data Cone

Real-world data is refined through layers of abstraction until it becomes concrete code.



Real-World Data

The raw problem as it exists outside the program

Abstract Data Types (ADTs)

A logical model: what operations are possible, independent of implementation

Data Structures

A concrete way to organize that data in memory

Fundamental Data Types

The primitives the language gives you (int, bool, references...)

Abstract Data Types (ADTs)

- An ***ADT*** represents the logical or conceptual level of the data.
- It consists of:
  1. A collection of data items in some ***Data Structure***
  2. Operations (***algorithms***) on the data items
- For example, a ***Stack*** supports retrieval in LIFO (Last In First Out) order. Basic operations are ***push*** and ***pop.*** It can be implemented using arrays (static or dynamic) or linked lists

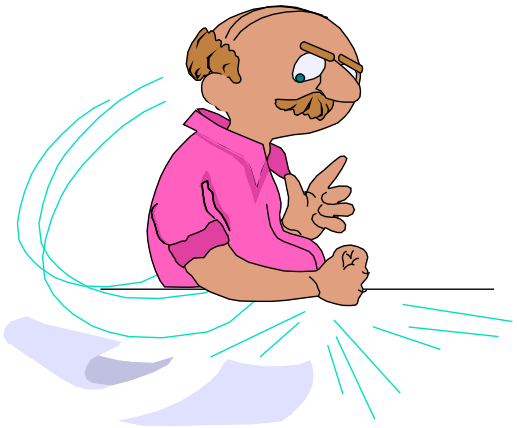
Data Structures

Definitions

- Ways to organize and store data
  - ✓ Data Storages
- Ways to access and manipulate the stored data.
  - ✓ Methods to access storages

Lessons

- Different data structures lead to different ways to solve a given problem. (algorithms).
- Different algorithms may give different efficiency (space and time).



Course outline

- Data Structures
  - ✓ Arrays, Linked Lists
  - ✓ Stacks, Queues
  - ✓ Sorting algorithms
  - ✓ Search Trees
  - ✓ Graphs
  - ✓ etc …

What is **Data**?

- Data is a collection of **raw facts and values**, such as **numbers**, **characters**, or **symbols**, that can be stored, processed, and organized.
- Data has no meaning until it is processed or interpreted.

Examples of Data:

- ➢ **Numeric** (e.g., 123, 45.6)
- ➢ **Textual** (e.g., "Hello", "CS203")
- ➢ **Boolean** (True/False)
- ➢ **Multimedia** (Images, Audio, Video)

Key Point:

- *Data becomes Information when processed to provide meaning and context.*



Data Types

Definition:

A data type defines a set of **values** and the **operations** that can be performed on them. It determines how data is represented in memory and how it can be manipulated.

Purpose:

- Classifies and organizes data into categories based on characteristics.
- Ensures data is stored and processed correctly.
- Provides rules for valid operations on data.

Examples:

- **Primitive Types:** integers, floating-point numbers, characters, booleans.
- **Composite Types:** strings, arrays, structures, classes.

What is a Data Structure?

Definition:

A data structure is a specialized **way of organizing, storing, and managing data** so that it can be used efficiently. n element may have a different typ

It groups data elements under one name

Key Points:

- Provides a systematic method to handle collections of data.
- Enables efficient data access, modification, and storage.
- **The choice of data structure directly affects the performance of algorithms.**

Examples:

- Arrays, Linked Lists, Stacks, Queues, Trees, Graphs

Abstract Data Types (ADT)

Definition:

An Abstract Data Type (ADT) specifies **what operations can be performed on data, without defining how these operations are implemented.**

Key Points:

- Focuses on the behavior of the data structure, not its implementation.
- Provides a logical description independent of the underlying code or storage method.

Example – Stack ADT:

- **Operations:** `push(x)`, `pop()`
- **Possible Implementations:** Array-based or Linked List-based

Classification of Data Structures

There are many classification techniques for data structures, each based on different criteria. These classifications help us understand how data is organized, stored, and managed, and guide us in choosing the most efficient structure for solving a specific problem.

Overview of Classification Methods:

The study of data structure classifications can be organized into five main categories:

1. **Based on fundamental nature and usage –** whether the structure is **primitive** or **non-primitive**.
2. **Based on Nature of Structure** – how data elements are logically arranged **linear** or **non-linear.**
3. **Based on Storage Method** – whether memory is **allocated statically** or **dynamically.**
4. **Based on Type of Elements** – whether elements are **homogeneous** or **heterogeneous**.
5. **Based on Memory Allocation** – whether memory is allocated **contiguously** or **non-contiguously.**

Classification Based on Usage

This classification depends on how data structures are applied in solving problems — whether they are used directly to store data or to support other operations.

1. **Primitive Data Structures:**
   - Basic structures provided by programming languages.
   - Used to represent simple data directly.

   Examples: Integers, Floats, Characters, Booleans.

2. **Non-Primitive Data Structures:**
   - Built on top of primitive types to handle more complex data.
   - Can be further divided into **Linear** (Arrays, Linked Lists, Stacks, Queues) and **Non-Linear** (Trees, Graphs).

Key Point:

Primitive structures are the foundation, while non-primitive structures enable efficient storage, manipulation, and problem-solving.

Classification Based on Nature of Structure

This classification describes how data elements are arranged within a data structure — either in a **sequential (linear) order** or a **hierarchical (non-linear)** form.

1. Linear Data Structures:
   - Data elements are arranged sequentially, one after another.
   - Elements have a clear order and are accessed in sequence.

   Examples: Arrays, Linked Lists, Stacks, Queues.

2. **Non-Linear Data Structures:**
   - Data elements are arranged in a non-sequential manner.
   - Elements are connected hierarchically or through complex relationships.

   Examples: Trees, Graphs.

Key Point:

Choosing the appropriate structure based on nature impacts the efficiency and complexity of data operations.

Classification Based on Storage Method

This classification is based on whether the size of a data structure is fixed or can change during program execution. It depends on how memory is allocated—either at **compile time (fixed)** or at **runtime (flexible).**

1.Static Data Structures:

- Memory size is fixed at compile time and cannot be changed during execution.
- Simpler to implement but less flexible.

Example: Arrays.

2.Dynamic Data Structures:

- Memory is allocated and managed at runtime, allowing growth or shrinkage as needed.
- More flexible and efficient for varying data requirements.

Examples: Linked Lists, Trees, Graphs.

Key Point:

Static storage ensures predictability, while dynamic storage provides adaptability and better memory utilization.

Classification Based on Type of Elements

This classification depends on whether the data elements stored in the structure are of the same type or of different types.

1. Homogeneous Data Structures:

- All elements are of the same data type.
- Simple and efficient for uniform data.

Examples: Arrays, Stacks, Queues.

2. Heterogeneous Data Structures:

- Elements can be of different data types.
- Provide flexibility for storing diverse information.

Examples: Structures, Classes, Linked Lists.

Key Point:

Homogeneous structures ensure uniformity, while heterogeneous structures offer flexibility to handle complex data.

Classification Based on Memory Allocation

This classification depends on how elements of a data structure are stored in memory — either in consecutive blocks or scattered with links.

1.Contiguous Memory Allocation:

- Elements are stored in consecutive memory locations.
- Commonly used for arrays.

Pros: Efficient random access.

Cons: Fixed size, potential fragmentation, not flexible for dynamic growth.

2.Non-Contiguous Memory Allocation:

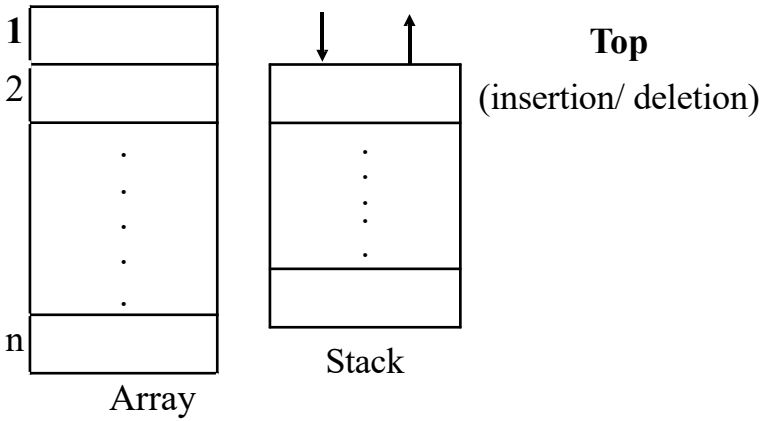
- Elements are stored in scattered memory locations, each linked by references (pointers).
- Commonly used for linked lists.

Pros: Supports dynamic size, efficient insertion/deletion.

Cons: Extra memory for pointers, slower traversal.

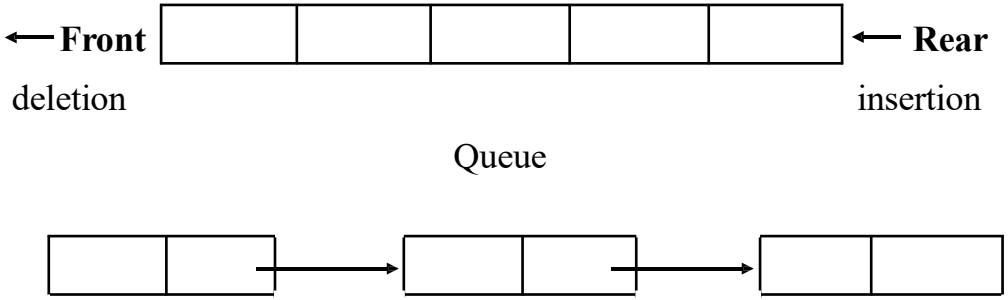
Key Point:

Contiguous allocation favors fast access, while non-contiguous allocation provides flexibility and efficient modifications.

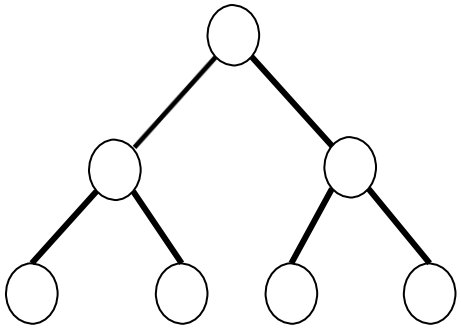


Array

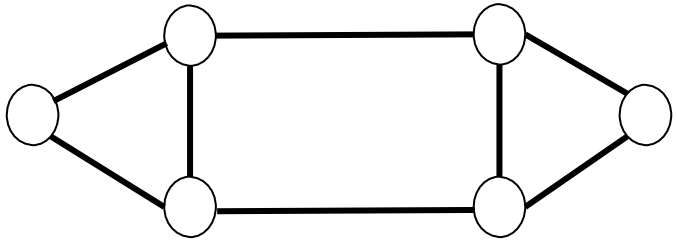
Stack



Queue



Tree



Graph

Data structure operations

The data structure operations grouped based on their primary purpose in the lifecycle of a data structure.

Creation:

1. **Creation/Initialization:** Establishing a new instance of the data structure.

Manipulation:

1. **Empty Check:** Assessing whether the data structure is devoid of elements.
2. **Full Check:** Evaluating if the data structure can accommodate additional elements.
3. **Size/Length:** Determining the quantity of elements stored in the data structure.
4. **Insertion:** Adding a new element to the data structure.
5. **Deletion:** Removing an existing element from the data structure.
6. **Search:** Locating a specific element within the data structure.
7. **Access/Retrieval:** Retrieving the value of a specific element in the data structure.
8. **Traversal:** Iterating through and processing each element in the data structure.

9. **Iteration:** Conducting a series of operations on each element of the data structure.
10. **Sorting:** Arranging the elements of the data structure in a specified order.
11. **Copy/Clone:** Generating a duplicate of the data structure.
12. **Merging:** Combining two or more data structures into a single entity.
13. **Splitting:** Segmenting a data structure into two or more smaller components.
14. **Concatenation:** Merging two data structures of the same type.
15. **Conversion:** Transforming the data structure from one representation to another (e.g., array to linked list).
16. **Comparison:** Assessing the equality or ordering of two data structures.
17. **Serialization/Deserialization:** Converting the data structure into a format suitable for storage or transmission, and vice versa.

Destruction:

1. **Destruction**: Removing the entire data structure from memory.

Overflow and Underflow

Both overflow and underflow errors can lead to unexpected behavior, program crashes, or other adverse effects, depending on how they're handled in the program.

- **Overflow:**

  This error occurs when attempting to add an element to a data structure that has reached its maximum capacity. It's commonly referred to as overflow because the data structure "overflows" with more elements than it can accommodate.

- **Underflow:**

  This error occurs when attempting to remove an element from a data structure that is already empty. It's referred to as underflow because the data structure "underflows" due to a lack of elements.

It's essential to handle these error conditions gracefully to prevent program failures and maintain the integrity of the data structures. This often involves implementing checks to ensure that operations like adding or removing elements are performed only when the data structure has sufficient capacity or contains elements, respectively.

Different data structures

Different data structures store elements in various ways, each with its own advantages and trade-offs in terms of **memory usage**, **access time**, and efficiency of operations. Here are some common data structures and how they store elements:

1. **Array**:
   - **An array stores elements in contiguous memory locations.**
   - **Elements are accessed using an index, which represents their position in the array.**
   - Arrays have constant-time access to elements (O(1)), but insertion and deletion operations may require shifting elements, resulting in linear-time complexity (O(n)).
2. **Linked List**:
   - **A linked list stores elements in nodes where each node contains a data element and a reference (pointer) to the next node in the sequence.**
   - sing the list from the head (start) to the tail (end).
   - d deletion at the beginning or end of the list (O(1)), but accessing elements at arbitrary positions requires traversing the list, resulting in linear-time complexity (O(n)).

3. Stack:

- **A stack stores elements in a Last-In-First-Out (LIFO) order.**
- Elements are added **(pushed)** and removed **(popped)** **from the top of the stack.**
- Stacks can be implemented using arrays or linked lists, with constant-time insertion and deletion operations (O(1)).

4. Queue:

- **A queue stores elements in a First-In-First-Out (FIFO) order.**
- Elements are added **(enqueued)** **at the rear (end) of the queue** and removed **(dequeued)** **from the front.**
- Queues can be implemented using arrays or linked lists, with constant-time insertion and deletion operations (O(1)).

5. Tree:

- **A tree stores elements in a hierarchical structure with a root node and child nodes.**
- Elements are organized into levels, with parent-child relationships between nodes.
- Trees can be binary (each node has at most two children) or non-binary (each node can have multiple children), with various traversal algorithms (e.g., in-order, pre-order, post-order) for accessing elements.

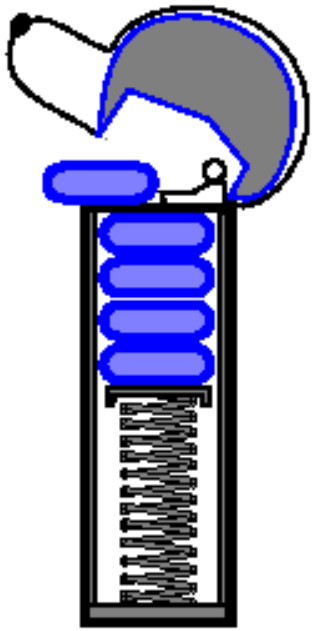
6. Graph:

- **A graph stores elements as vertices (nodes) and edges (connections between vertices).**
- Elements can be connected in any arbitrary manner, allowing for complex relationships between data elements.
- Graphs can be directed (edges have a specific direction) or undirected (edges have no direction), and they can be represented using various data structures such as adjacency lists or adjacency matrices.

These are just a few examples of common data structures and how they store elements. Each data structure has its own characteristics and is suitable for different types of applications and operations. **Choosing the right data structure is essential for designing efficient algorithms and software solutions.**

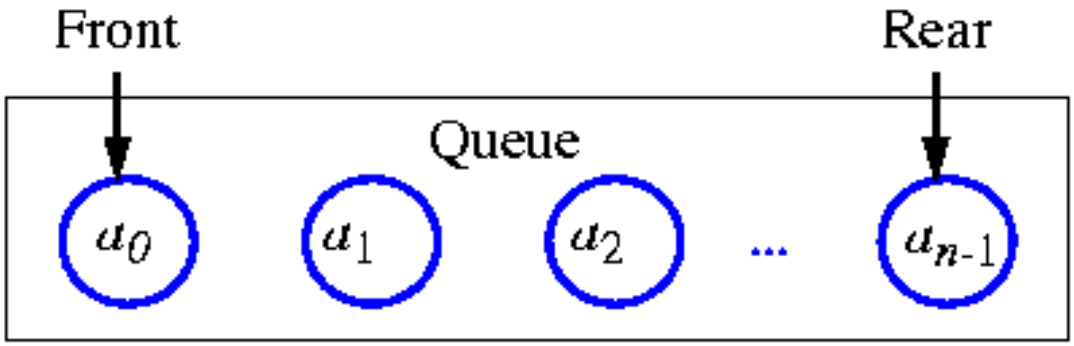
Stack

- A container of objects that are inserted and removed according to the last-in-first-out (LIFO) principle.
- Only the last (the most recently inserted) object can be removed.



Queue

- Differs from a stack in that its insertion and removal follows the first-in-first-out (FIFO) principle.
- The element which has been in the queue the longest may be removed.



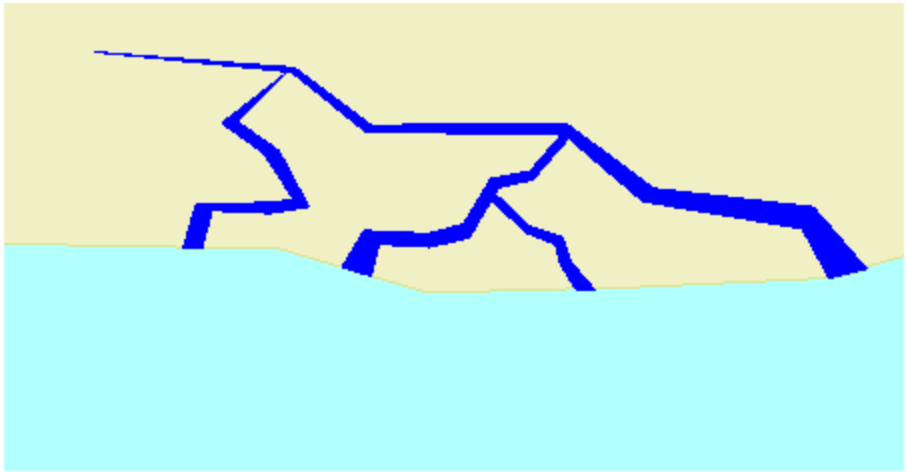
List

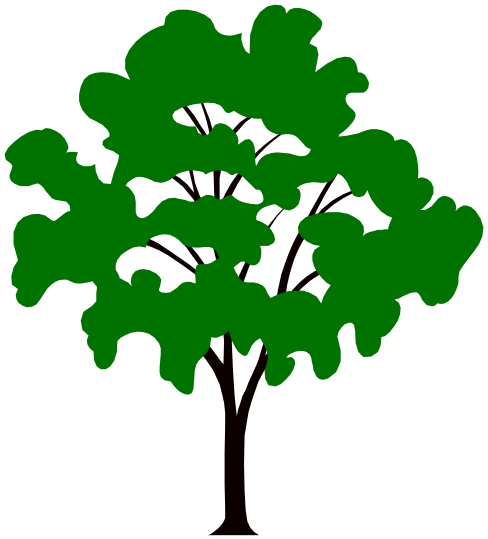
- A collection of linearly arranged element (a linear order).
- Provides methods for accessing, inserting, and removing arbitrary elements.
- Notion of position, before and after.

  - Stacks and queues are a restricted form of a sequence.
  - Example,
    - ✓ A,B,C,D,E,F
    - ✓ a_1, a_2, a_3,…

Tree

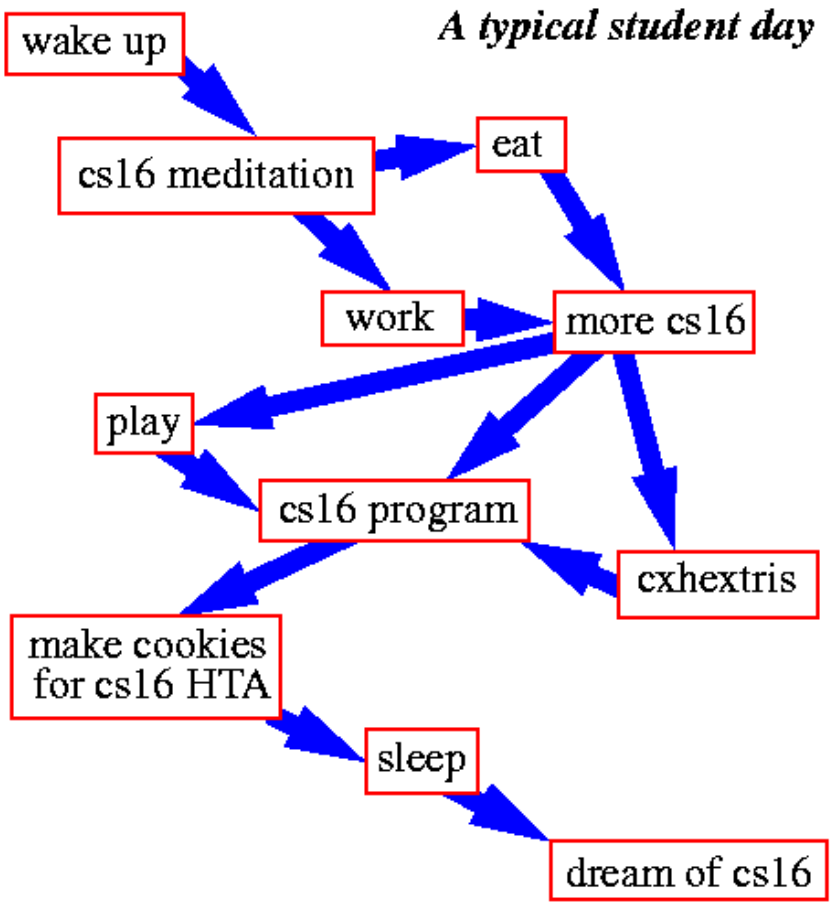
- A collection of objects arranged in a hierarchical fashion.
- E.g., organization of a corporation, a table of content, dos/unix file systems, family tree.
- Notion of parents and children, root and leaves.





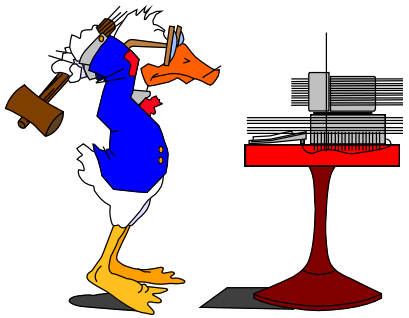
Graphs

- Not only physical connectivity, but also logical relationship.

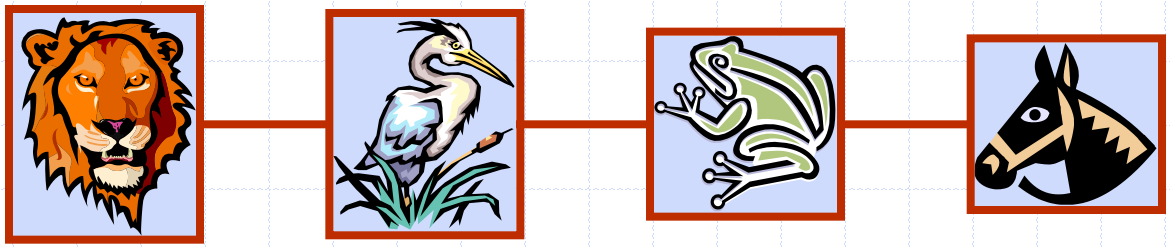


Algorithms and Applications

- Every computer software uses some collections of data structures.
- We will study algorithms to efficiently solve problems using various data structures.
- Proof techniques for correctness or efficiency.



Array



Overview and Reading

- Basic Elementary Data Structures

- Array
- Linked Lists
  - Singly linked lists
  - Doubly linked lists
  - Circular linked lists
- These are used for more advanced data structures later

Linear data structures

Linear data structures are those that organize and store data elements sequentially, where each element is connected to its predecessor and/or successor. In other words, the elements are arranged in a linear order, and each element (except possibly the first and last) has a **unique** predecessor and successor.

Key characteristics of linear data structures include:

1. **Sequential Ordering**: Elements are arranged in a specific order, typically from one end to another, without any branching or hierarchical structure.
2. **Traversal**: Accessing elements in a linear data structure involves traversing the structure sequentially from one end to another. This sequential access allows for efficient iteration over all elements.
3. **Operations**: Basic operations performed on linear data structures include insertion (adding elements), deletion (removing elements), searching (finding elements), and traversal (visiting all elements).

Common examples of linear data structures include:

1. **Arrays:**

   An **array** is a **collection of elements** of the **same data type**, stored in **contiguous memory locations.**

   - Each element is identified by an **index**, which allows **constant-time access (O(1))** to any element.

   Key Point:

   Arrays are highly efficient for **random access**, but their **size is fixed once created**.

2. **Linked Lists:**

   A **linked list** is a **collection of nodes**, where each node contains:

   - A **data value**, and
   - A **reference (pointer)** to the **next node** in the sequence.

   Unlike arrays, linked lists **do not require contiguous memory**, allowing for **dynamic resizing** and **efficient insertion and deletion** operations.

   Key Point:

   Linked lists are **flexible in size**, but **accessing elements is sequential (O(n))**, not constant-time.

3. Stacks:

A **stack** is a **linear data structure** that follows the **Last-In-First-Out (LIFO)** principle. Elements are **added (pushed)** and **removed (popped)** from the **same end**, called the **top**.

Common Applications:

- Managing **function calls** (call stack)
- **Expression evaluation** (e.g., postfix or prefix)
- **Backtracking algorithms** (e.g., maze solving, undo operations)

Key Point:

The **last element inserted** is the **first one removed**.

4. Queues:

A **queue** is a **linear data structure** that follows the **First-In-First-Out (FIFO)** principle. Elements are **added at the rear (enqueue)** and **removed from the front (dequeue)**.

Common Applications:

- **Process scheduling** in operating systems
- **Breadth-First Search (BFS)** in graph algorithms
- **Buffering** in data streams or I/O operations

Key Point:

The **first element inserted** is the **first one removed**.

Array and Linked list

Arrays and **Linked lists** are often grouped together because they offer flexibility in how elements are accessed, added, and removed, without imposing specific constraints. In contrast, **Stacks** and **Queues** enforce specific rules or constraints on how elements are accessed, added, and removed.

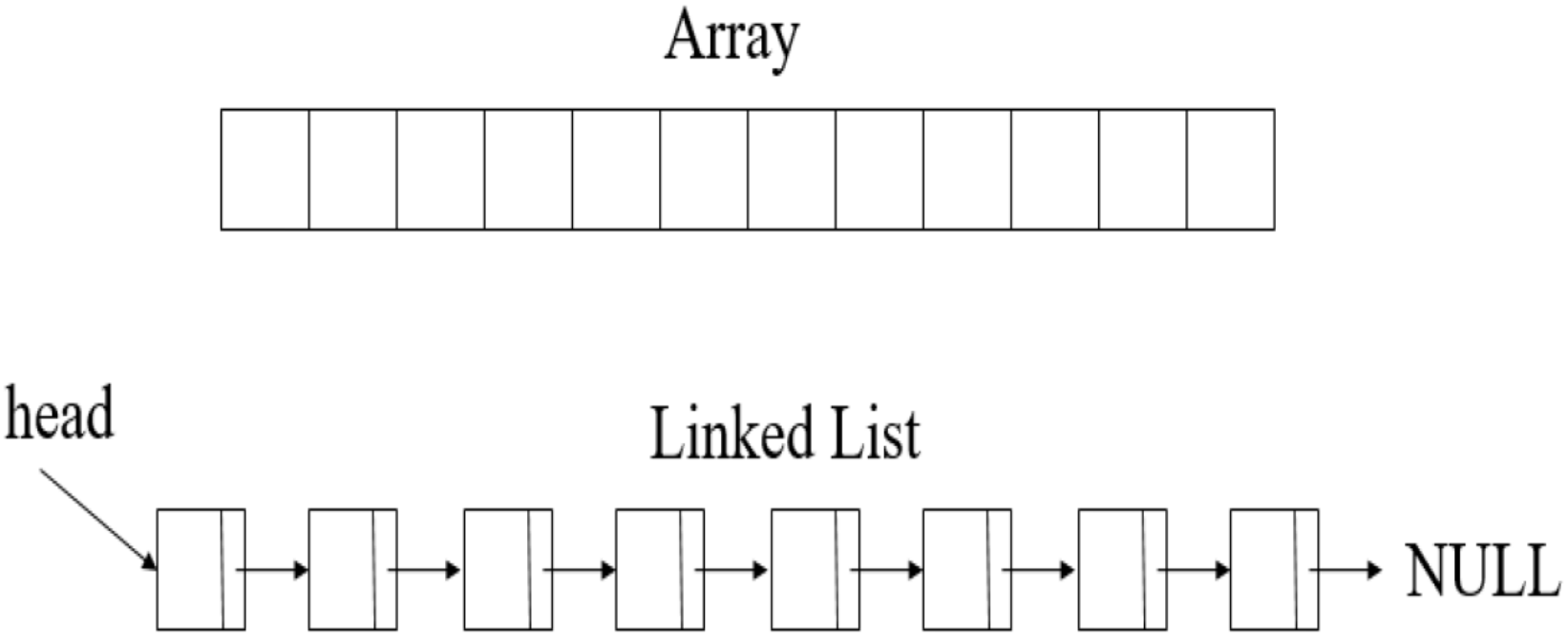
However, they have distinct characteristics and trade-offs:

Arrays:

1. Contiguous memory allocation.
2. Constant-time access to elements by index.
3. Fixed size once allocated (in most languages).
4. **Costly resizing operations (typically O(n)).**
5. Efficient for random access and iteration.

Linked Lists:

1. Non-contiguous memory allocation, using pointers or references to link elements.
2. Dynamic size, allowing for efficient insertion and deletion operations.
3. Linear-time access to elements (O(n)) as traversal is required.
4. No wasted memory due to resizing.
5. Less efficient for random access compared to arrays.

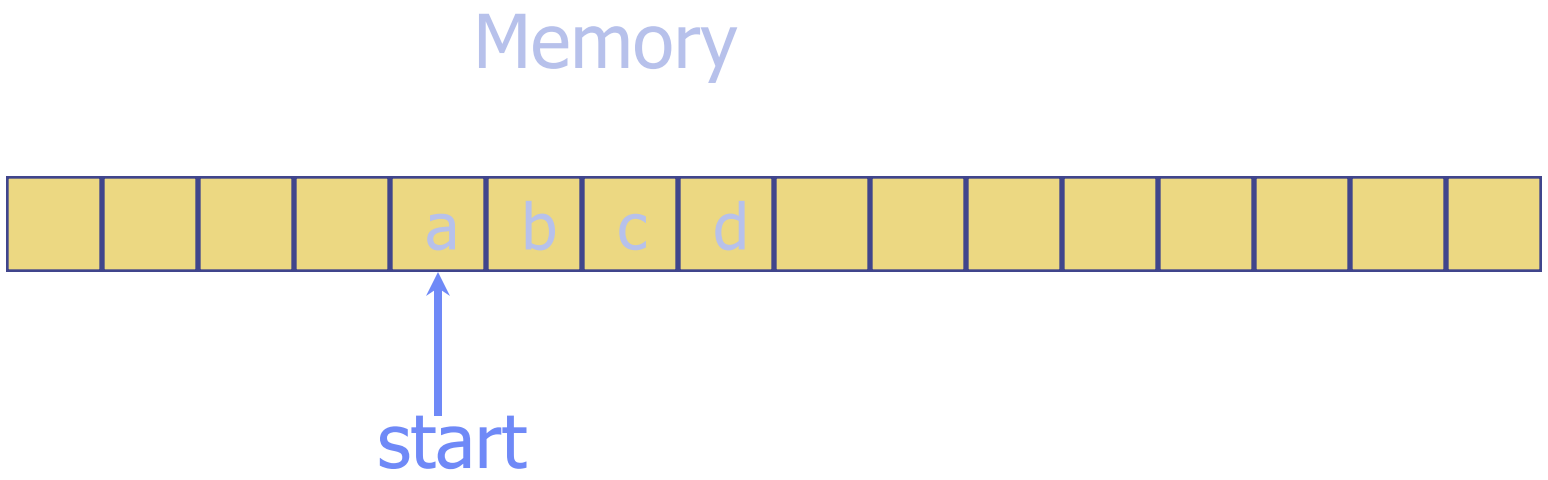

Array
head
Linked List
NULL

Array vs. Linked List

They each have their advantages and disadvantages depending on the specific requirements of the application.

- **Advantages of Arrays:**
  - ➢ Can directly select any element.
  - ➢ No memory wasted for storing pointers.
- **Disadvantages of Arrays:**
  - ➢ Fixed size (cannot grow or shrink dynamically).
  - ➢ Need to shift elements to insert an element to the middle.
  - ➢ Memory wasted due to unused elements.
- **Advantages of Linked Lists:**
  - ➢ Dynamic size (can grow and shrink as needed).
  - ➢ No need to shift elements to insert into the middle.
  - ➢ Size can exactly match the number of elements (no wasted memory).
- **Disadvantages of Linked Lists:**
  - ➢ Cannot directly select any element (need to follow pointers)
  - ➢ Extra memory usage for storing pointers.

Array (§ 3.1)



- Storing data in a sequential memory locations
- Access each element using integer index
- Very basic, popular, and simple

Array: Problems

- New insertion and deletion: difficult
  - Need to shift to make space for insertion
  - Need to fill empty positions after deletion

- Why don't we connect all elements just "logically" not "physically"?
  - Linked List

Array: operations

- Inserting an item at first
- Inserting an item at last
- Inserting in a specific position
- display()
- getSize()
- getlength()
- find(int value)
- deleteElement(int position)
- resize(int newSize)
- Merge(Array array2)

```
6 references
class MyArray
{
    9 references
    private int size = 0;
    21 references
    private int length = 0;
    19 references
    private int[] elements;

    // Constructor
    2 references
    public MyArray(int arraySize)
    {
        size = arraySize;
        length = 0;
        elements = new int[size];
    }
```

```
// Fill the array
2 references
public void Insert()
{
    Console.Write("How many elements do you want to set: ");
    int numItems = int.Parse(Console.ReadLine());

    if (numItems <= (size - length))
    {
        for (int i = 1; i <= numItems; i++)
        {
            Console.Write("Enter item No. " + i + " : ");
            elements[length++] = int.Parse(Console.ReadLine());
        }
    }
    else
    {
        Console.WriteLine(
            "Error: Not enough space to insert the elements you want");
    }
}
```

```
// Append an element
1 reference
public void Insert(int value)
{
    if (length < size)
    {
        elements[length++] = value;
    }
    else
    {
        Console.WriteLine(
            "Error: Not enough space to insert the element you want");
    }
}
```

```
// Insert at a specific position
1 reference
public void Insert(int position, int value)
{
    if (length < size && position >= 0)
    {
        if (position <= length)
        {
            // Shift elements to the right
            for (int i = length; i > position; i--)
            {
                elements[i] = elements[i - 1];
            }

            elements[position] = value;
            length++;
        }
        else
        {
            Console.WriteLine(
                "Error: The position you want to insert into " +
                "is greater than the length of this array");
        }
    }
    else
    {
        Console.WriteLine(
            "Error: Not enough space to insert the element you want");
    }
}
```

```
// Display array elements
2 references
public void Display()
{
    Console.WriteLine("Displaying the elements of this array:");

    if (length != 0)
    {
        for (int i = 0; i < length; i++)
        {
            Console.Write(elements[i] + "\t");
        }

        Console.WriteLine();
    }
    else
    {
        Console.WriteLine("The array is empty.");
    }
}
```

```
// Get array size
8 references
public int GetSize()
{
    return size;
}

// Get array length
8 references
public int GetLength()
{
    return length;
}

// Search for an element
1 reference
public int Find(int value)
{
    int position = -1;

    for (int i = 0; i < length; i++)
    {
        if (elements[i] == value)
        {
            position = i;
            break;
        }
    }

    return position;
}
```

```
// Delete an element
1 reference
public void DeleteElement(int position)
{
    if (position < length && position >= 0)
    {
        // Shift elements to the left
        for (int i = position; i < length - 1; i++)
        {
            elements[i] = elements[i + 1];
        }

        length--;

        // Clear the last element
        elements[length] = 0;
    }
    else
    {
        Console.WriteLine(
            "Error: The position you entered is not valid");
    }
}
```

```
// Resize the array
1 reference
public void Resize(int newSize)
{
    if (newSize >= length)
    {
        int[] oldElements = elements;

        elements = new int[newSize];

        // Copy old elements
        for (int i = 0; i < length; i++)
        {
            elements[i] = oldElements[i];
        }

        size = newSize;
    }
    else
    {
        Console.WriteLine(
            "Error: The length of the existing array " +
            "is greater than the new size");
    }
}
```

```
    // Merge another array with this array
    1 reference
    public void Merge(MyArray array2)
    {
        int newSize = size + array2.GetSize();

        int[] oldElements = elements;

        elements = new int[newSize];

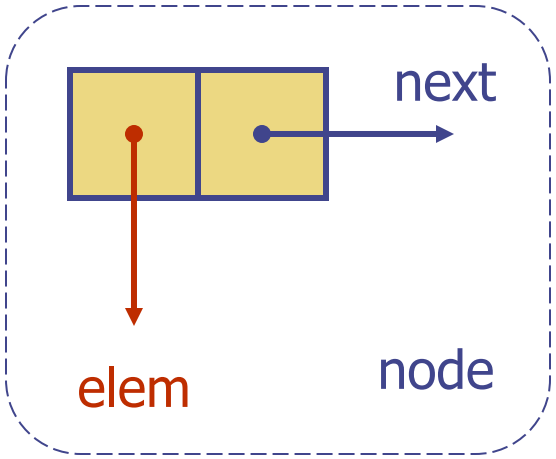
        // Copy elements of the first array
        for (int i = 0; i < length; i++)
        {
            elements[i] = oldElements[i];
        }

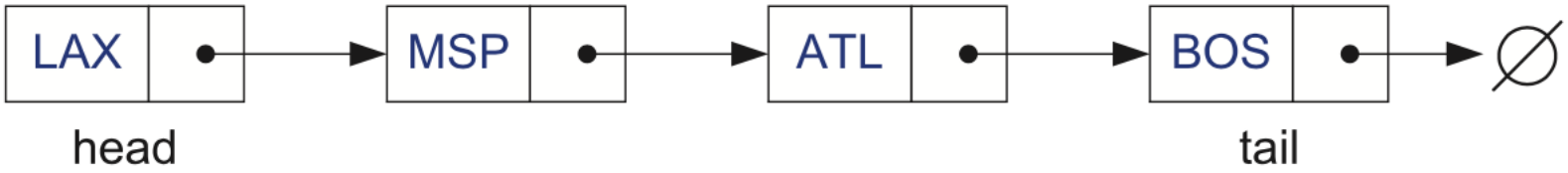
        // Copy elements of the second array
        for (int i = 0; i < array2.GetLength(); i++)
        {
            elements[length++] = array2.elements[i];
        }

        size = newSize;
    }
}
```

Singly Linked List (§ 3.2)

- A singly linked list is a concrete data structure consisting of a sequence of nodes
- Each node stores
  - element
  - link to the next node





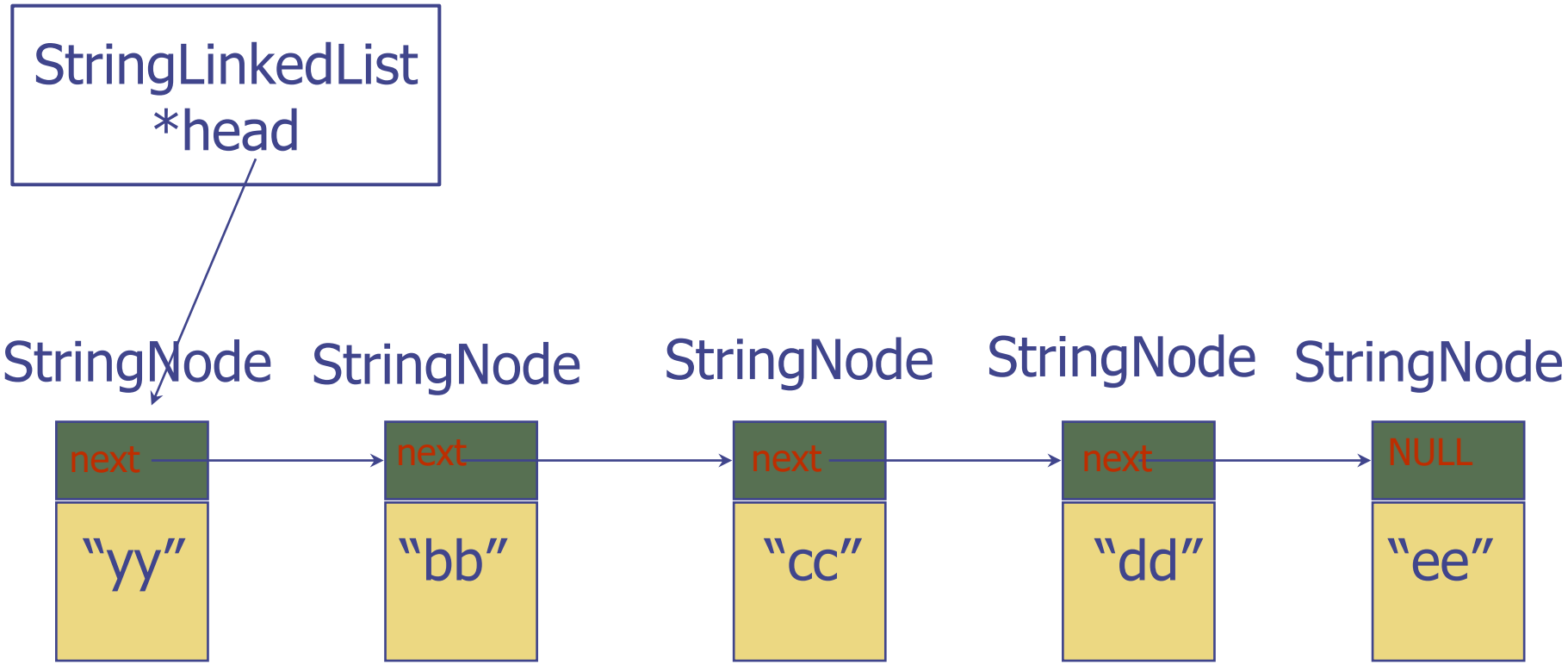
Example: Linked list of strings

```
class StringNode {                          // a node in a list of strings
private:
  string elem;                              // element value
  StringNode* next;                         // next item in the list

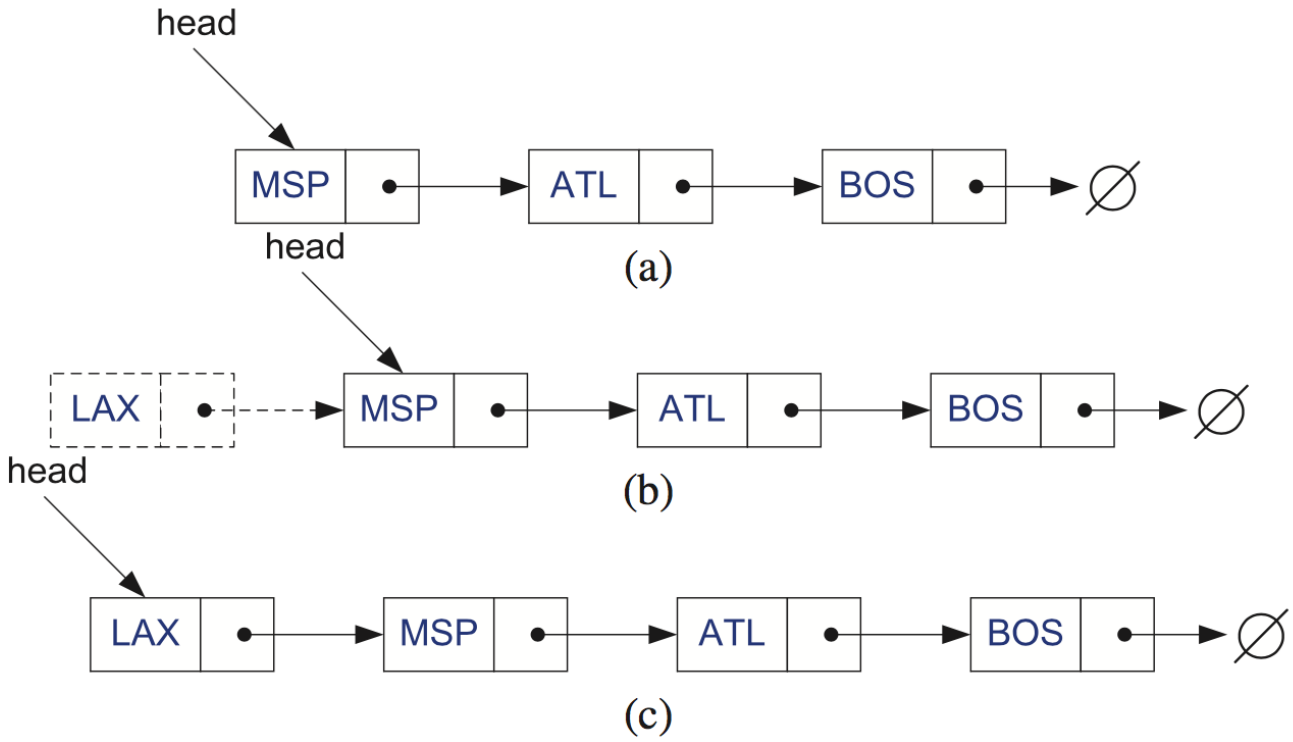
  friend class StringLinkedList;            // provide StringLinkedList access
};
```

```
class StringLinkedList {                    // a linked list of strings
public:
  StringLinkedList();                       // empty list constructor
  ~StringLinkedList();                      // destructor
  bool empty() const;                       // is list empty?
  const string& front() const;              // get front element
  void addFront(const string& e);           // add to front of list
  void removeFront();                       // remove front item list
private:
  StringNode* head;                         // pointer to the head of list
};
```

Singly Linked List of Strings: Picture

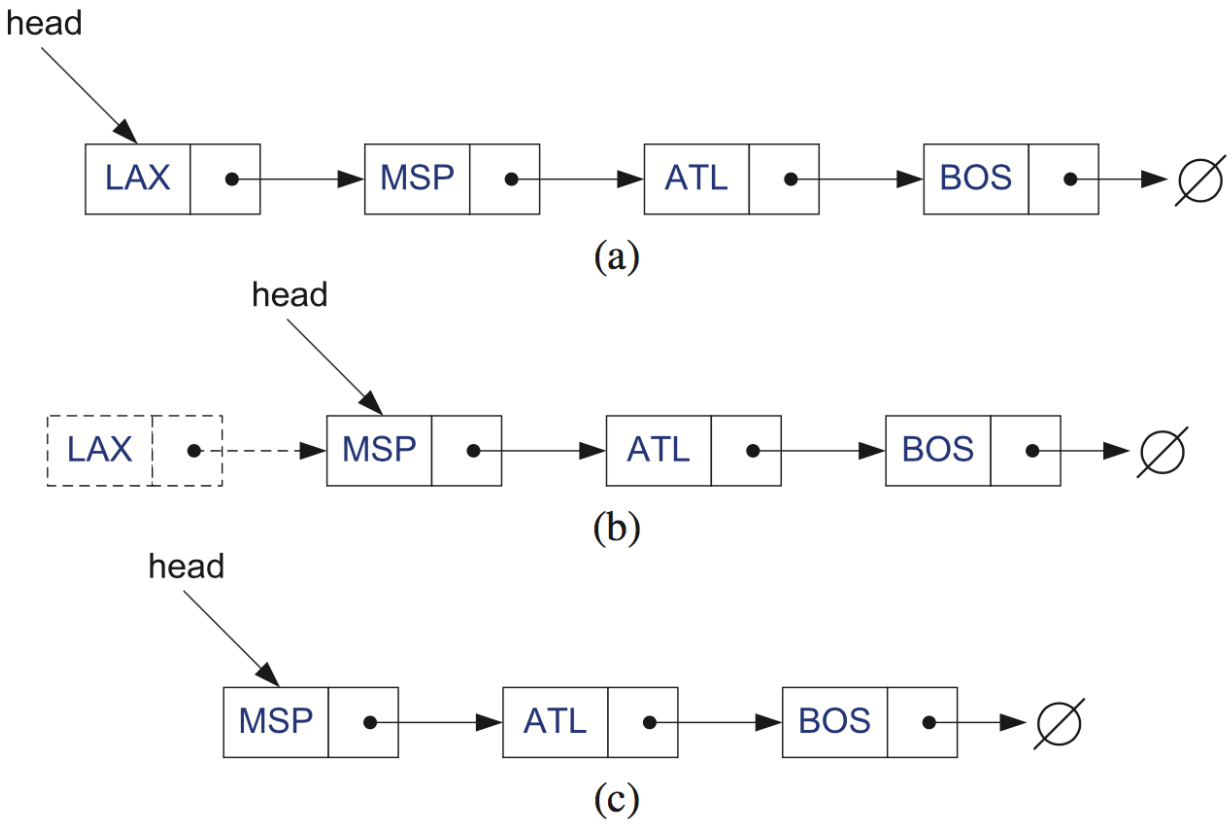


Inserting at the Head



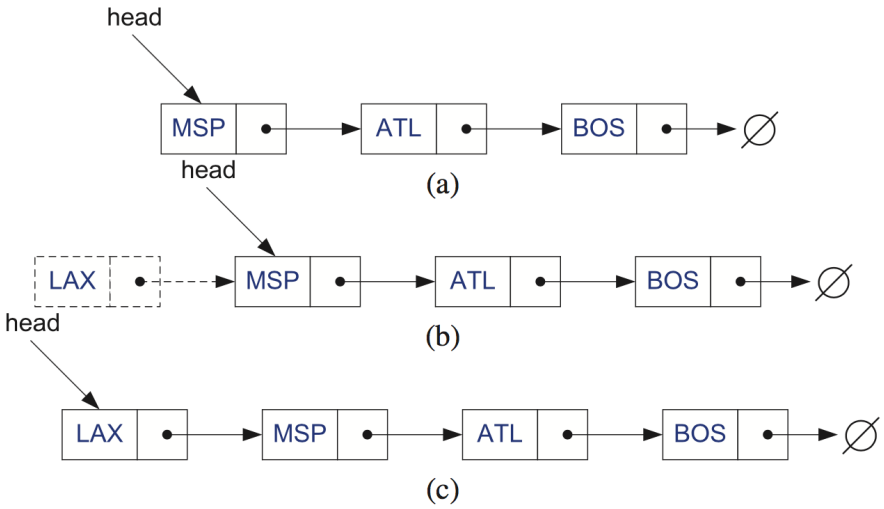
1. Allocate a new node
2. Insert a new element
3. Have the new node point to the old head
4. Update head to point to new node

Removing at the Head

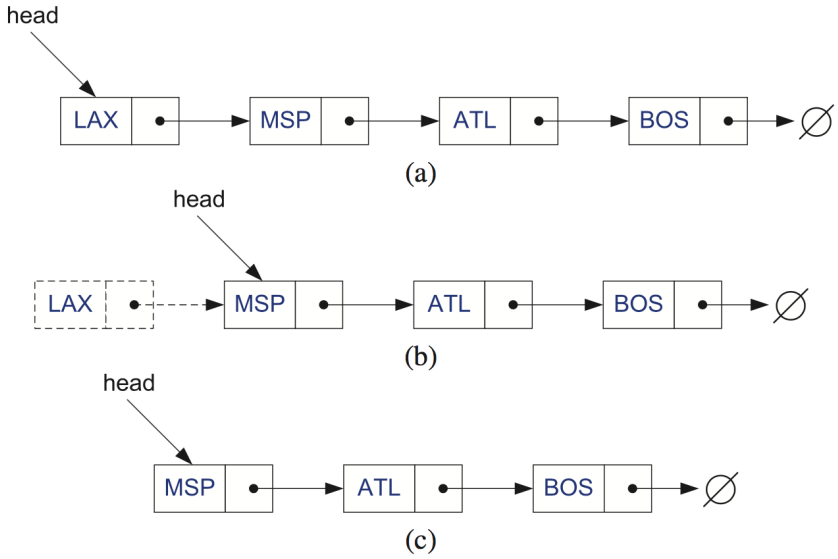


1. Update head to point to next node in the list
2. Allow garbage collector to reclaim the former first node (typically done by calling "delete" in C++)

Let's make codes




```
void StringLinkedList::addFront(const string& e) {
}
```




```
void StringLinkedList::removeFront() {
}
```

Inserting at the Tail and Removing at the Tail

1. Allocate a new node
2. Insert new element
3. Have new node point to null
4. Have old last node point to new node
5. Update tail to point to new node

Insertion at the tail

1. …
2. …
3. …
4. …

Removal at the tail

”Generic” Singly Linked Lists: Template

```
class Node
{
    9 references
    public int Data;
    21 references
    public Node Next;
    3 references
    public Node()
    {
        Next = null;
    }
}
```

```
3 references
class MyLinkedList
{
    15 references
    public Node Head;

    1 reference
    public MyLinkedList()
    {
        Head = null;
    }

    3 references
    public bool IsEmpty()
    {
        return Head == null;
    }
```

”Generic” Singly Linked Lists: Template

```
public void InsertFirst(int value)
{
    Node node1 = new Node();
    node1.Data = value;
    node1.Next = Head;
    Head = node1;
}
```

```
public void InsertLast(int value)
{
    if (IsEmpty())
    {
        InsertFirst(value);
    }
    else
    {
        Node node1 = new Node();
        node1.Data = value;
        node1.Next = null;

        Node ptr = Head;

        while (ptr.Next != null)
        {
            ptr = ptr.Next;
        }

        ptr.Next = node1;
    }
}
```

”Generic” Singly Linked Lists: Template

```
public void InsertBefore(int value, int before)
{
    if (Find(before))
    {
        if (Head.Data == before)
        {
            InsertFirst(value);
        }
        else
        {
            Node node1 = new Node();
            node1.Data = value;

            Node ptr = Head;

            while (ptr.Next.Data != before)
            {
                ptr = ptr.Next;
            }

            node1.Next = ptr.Next;
            ptr.Next = node1;
        }
    }
    else
    {
        Console.WriteLine(
            "The element " + before + " is not found");
    }
}
```

”Generic” Singly Linked Lists: Template

```
public void Delete(int value)
{
    if (Find(value))
    {
        if (Head.Data == value)
        {
            Node deptr = Head;
            Head = Head.Next;
            deptr = null;
        }
        else
        {
            Node ptr = Head;

            while (ptr.Next.Data != value)
            {
                ptr = ptr.Next;
            }

            Node deptr = ptr.Next;
            ptr.Next = ptr.Next.Next;
            deptr = null;
        }
    }
    else
    {
        Console.WriteLine(
            "The element " + value + " is not found");
    }
}
```

”Generic” Singly Linked Lists: Template

```
public void Display()
{
    if (IsEmpty())
    {
        Console.WriteLine("The list is empty");
    }
    else
    {
        Node ptr = Head;

        Console.WriteLine(
            "Displaying the elements of this list");

        while (ptr != null)
        {
            Console.Write(ptr.Data + "\t");
            ptr = ptr.Next;
        }

        Console.WriteLine();
    }
}
```

”Generic” Singly Linked Lists: Template

```
public int Count()
{
    int countr = 0;
    Node ptr = Head;

    while (ptr != null)
    {
        countr++;
        ptr = ptr.Next;
    }

    return countr;
}
```

```
public bool Find(int value)
{
    Node ptr = Head;

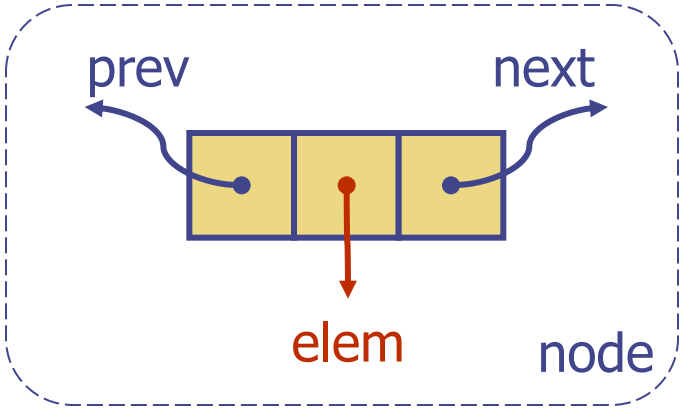
    while (ptr != null)
    {
        if (ptr.Data == value)
        {
            return true;
        }

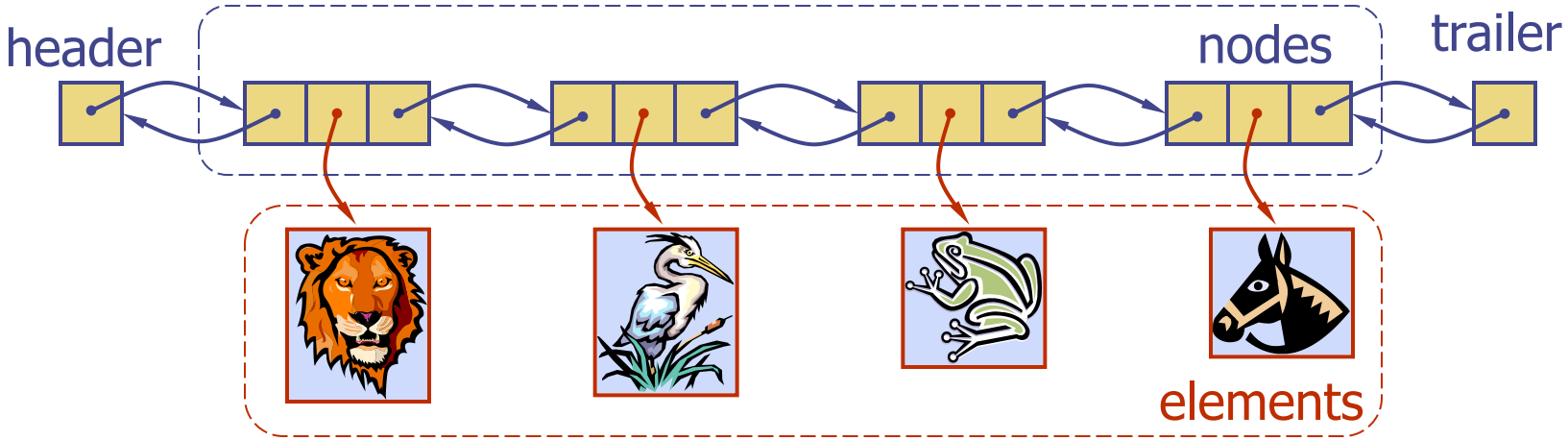
        ptr = ptr.Next;
    }

    return false;
}
```

Doubly Linked List (§ 3.3)

- Singly Linked List
  - Not easy to remove an elem. at the tail (or any other node)
- Trailer: Dummy sentinel
- Previous link





C++ Implementation: Class Design

```
typedef string Elem;
class DNode {
private:
  Elem elem;
  DNode* prev;
  DNode* next;
  friend class DLinkedList;
};
```

```
class DLinkedList {
public:
  DLinkedList();
  ~DLinkedList();
  bool empty() const;
  const Elem& front() const;
  const Elem& back() const;
  void addFront(const Elem& e);
  void addBack(const Elem& e);
  void removeFront();
  void removeBack();
private:
  DNode* header;
  DNode* trailer;
protected:
  void add(DNode* v, const Elem& e);
  void remove(DNode* v);
};
```

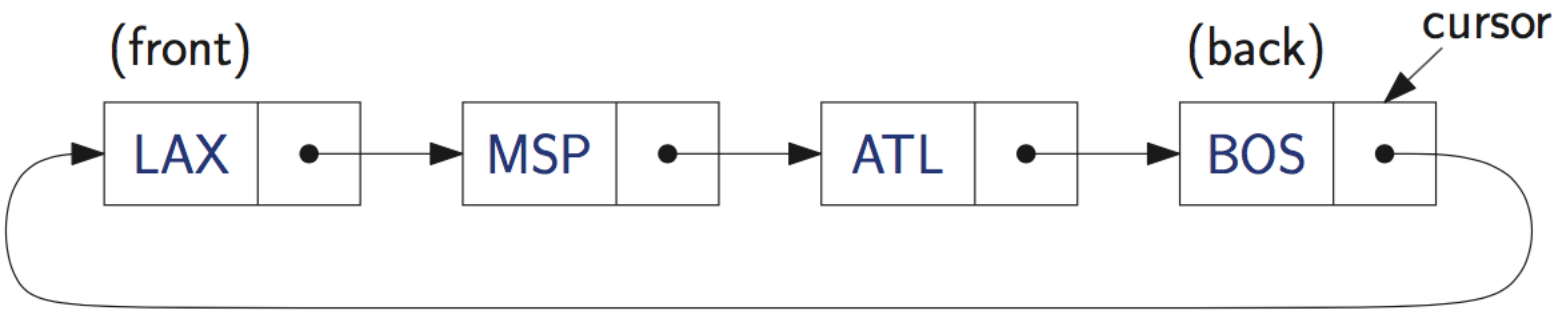
Constructor and Destructor (Don't forget!)

```
DLinkedList::DLinkedList() {                    // constructor
  header = new DNode;                           // create sentinels
  trailer = new DNode;
  header->next = trailer;                       // have them point to each other
  trailer->prev = header;
}

DLinkedList::~DLinkedList() {                   // destructor
  while (!empty()) removeFront();               // remove all but sentinels
  delete header;                                // remove the sentinels
  delete trailer;
}
```

Circular Linked List (§ 3.3)

- A kind of Singly Linked List
- Rather than having a head or a tail, it forms a cycle
- Cursor
  - A virtual starting node
  - This can be varying as we perform operations



C++ Implementation

```
typedef string Elem;
class CNode {
private:
  Elem elem;
  CNode* next;

  friend class CircleList;
};
```

```
class CircleList {
public:
  CircleList();
  ~CircleList();
  bool empty() const;
  const Elem& front() const;
  const Elem& back() const;
  void advance();
  void add(const Elem& e);
  void remove();
private:
  CNode* cursor;
};
```

What is advance()?

Summary

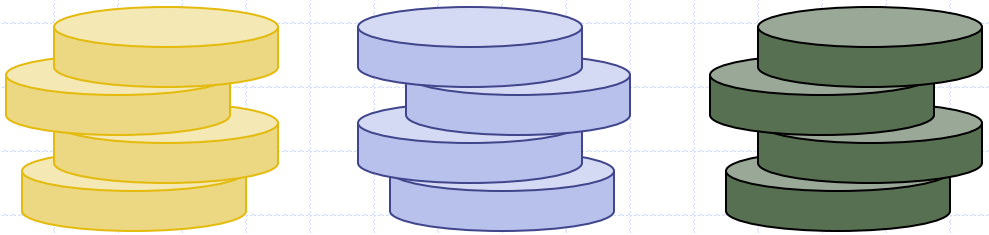
- Array and Lists
  - A simple data structure to store multiple elements (of the same type)

- Array
- Singly Linked Lists
- Doubly Linked Lists
- Circular Linked Lists

- Key Question
  - For each of the operations, how efficiently does each data structure perform the operation?

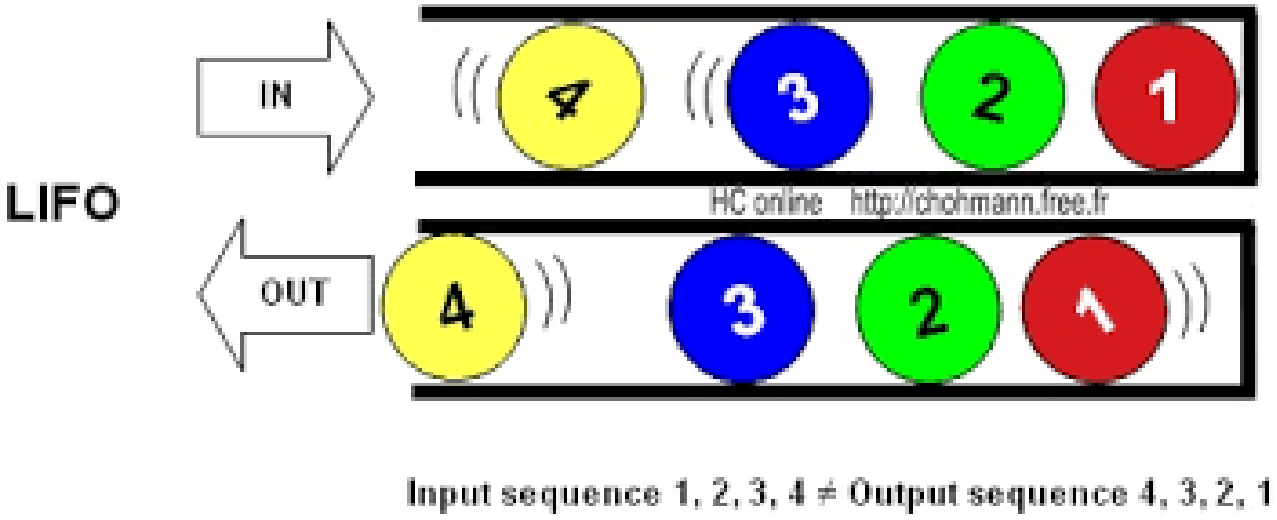
Questions?

Stacks



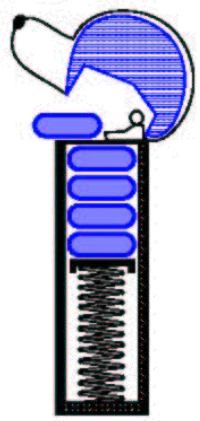
Overview

- Last-In-First-Out Data Structure



The Stack ADT

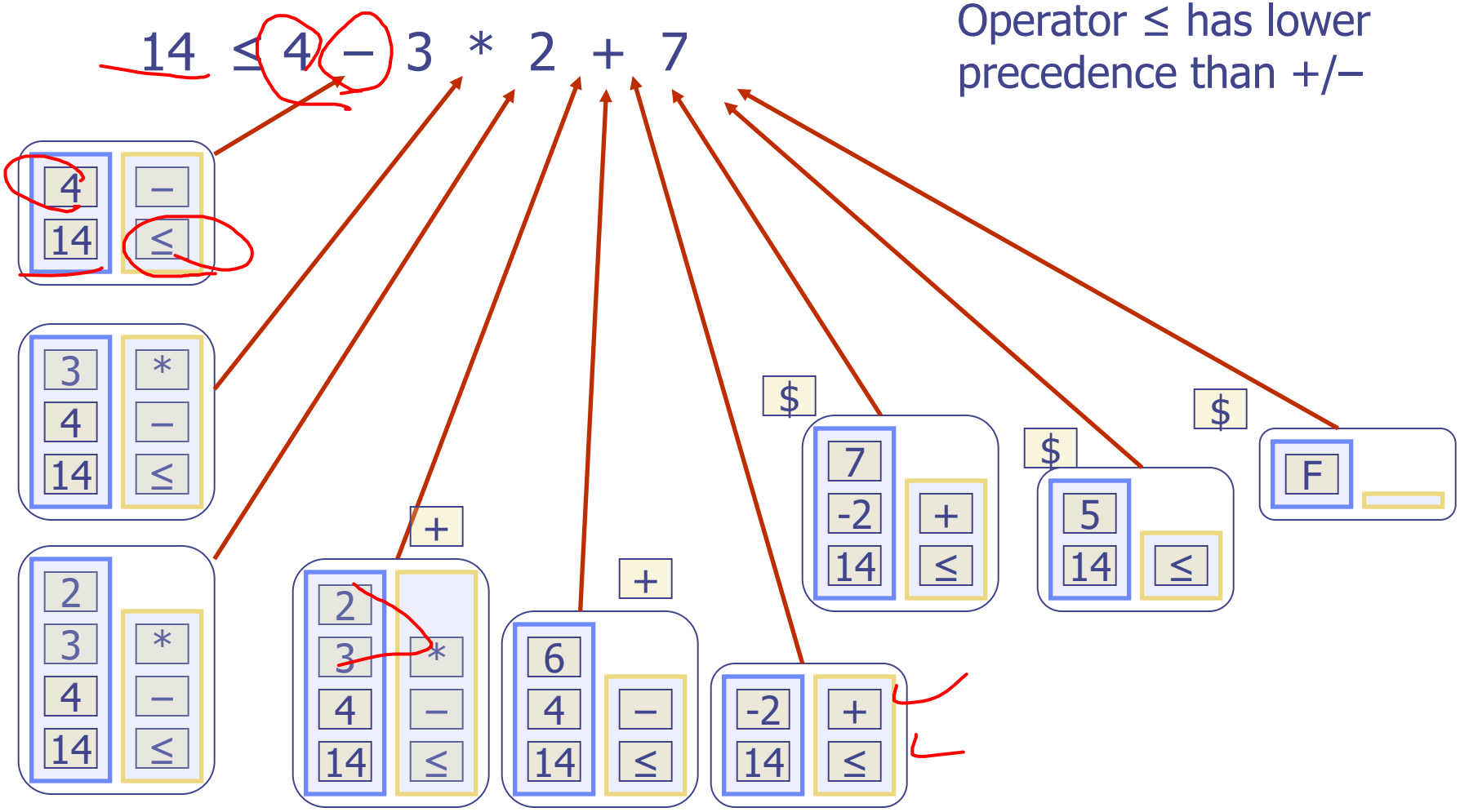
- The Stack ADT stores arbitrary objects
- Insertions and deletions follow the last-in first-out scheme
- Think of a spring-loaded plate dispenser
- **Main stack operations**:
  - push(object): inserts an element
  - object pop(): removes the last inserted element
- **Auxiliary stack operations**:
  - object top(): returns the last inserted element without removing it
  - integer size(): returns the number of elements stored
  - boolean empty(): indicates whether no elements are stored



Applications of Stacks

- Direct applications
  - Page-visited history in a Web browser
  - Undo sequence in a text editor
  - Chain of method calls in the C# run-time system
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

Example: Algorithm on an Example Expression



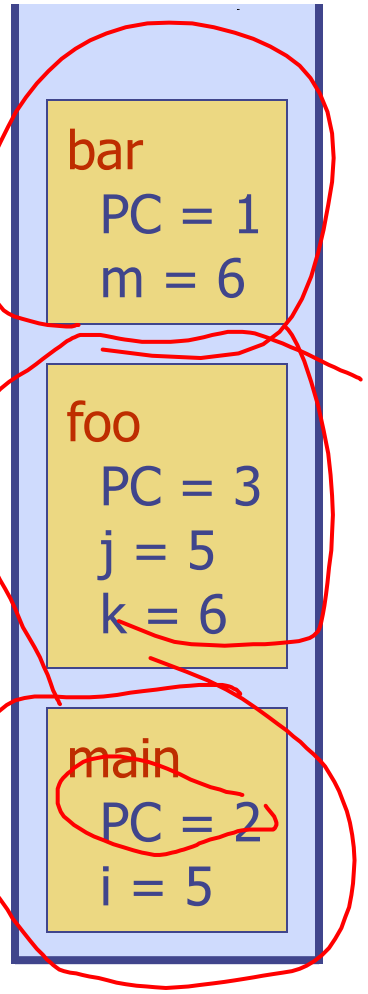
Example: Run-Time Stack

- The run-time system keeps track of the chain of active functions with a stack
- When a function is called, the system pushes on the stack a frame containing
  - Local variables and return value
  - Program counter, keeping track of the statement being executed
- When the function ends, its frame is popped from the stack and control is passed to the function on top of the stack
- Allows for recursion
- PC: Program Counter

```
main() {
  int i = 5;
  foo(i);
}

foo(int j) {
  int k;
  k = j+1;
  bar(k);
}

bar(int m) {
  …
}
```



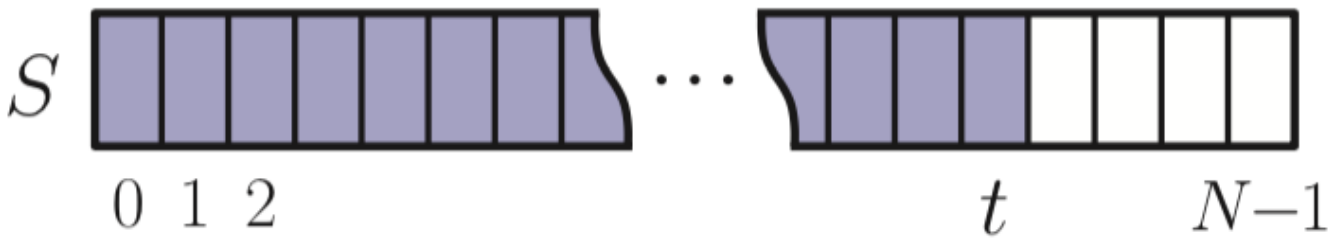
Example: Parentheses Matching

- Each “(”, “{”, or “[” must be paired with a matching “)”, “}”, or “[”
  - correct: ( )(( )){([( )])}
  - correct: ((( )(( )){([( )])}))
  - incorrect:)(( )){([( )])}
  - incorrect: ({[ ])}
  - incorrect: (

- Good Programmer
  - Someone who thinks that stack is a good data structure for the above task

Example Implementation: Array-based Stack

- A simple way of implementing the Stack ADT uses an array
- We add elements from left to right
- A variable keeps track of the index of the top element



Example Implementation: Array-based Stack

- A simple way of implementing the Stack ADT
- Add elements from left to right
- A variable keeps track of the index of the top element
- The array storing the stack elements may become full
  - A push operation will then throw a StackFull exception
  - Limitation of the array-based implementation
  - Not intrinsic to the Stack ADT

Algorithm size():
    return $t + 1$
Algorithm empty():
    return $(t < 0)$
Algorithm top():
    if empty() **then**
        throw StackEmpty exception
    return $S[t]$
Algorithm push(e):
    if size() $= N$ **then**
        throw StackFull exception
    $t \leftarrow t + 1$
    $S[t] \leftarrow e$
Algorithm pop():
    if empty() **then**
        throw StackEmpty exception
    $t \leftarrow t - 1$

Example

* indicates top

```
ArrayStack<int> A;                          // A = [ ], size = 0
A.push(7);                                  // A = [7*], size = 1
A.push(13);                                 // A = [7, 13*], size = 2
cout << A.top() << endl; A.pop();           // A = [7*], outputs: 13
A.push(9);                                  // A = [7, 9*], size = 2
cout << A.top() << endl;                    // A = [7, 9*], outputs: 9
cout << A.top() << endl; A.pop();           // A = [7*], outputs: 9
ArrayStack<string> B(10);                   // B = [ ], size = 0
B.push("Bob");                              // B = [Bob*], size = 1
B.push("Alice");                            // B = [Bob, Alice*], size = 2
cout << B.top() << endl; B.pop();           // B = [Bob*], outputs: Alice
B.push("Eve");                              // B = [Bob, Eve*], size = 2
```

Performance and Limitations

- Performance
  - Let n be the number of elements in the stack
  - The space used is $O(n)$
  - Each operation runs in time $O(1)$

- Limitations
  - The maximum size of the stack must be defined a priori and cannot be changed
  - Trying to push a new element into a full stack causes an implementation-specific exception

Linked List-based Stack

```
class Node
{
    4 references
    public int Data;
    6 references
    public Node Next;

    1 reference
    public Node()
    {
        Next = null;
    }
}
```

```
class MyStack
{
    10 references
    public Node Top;

    1 reference
    public MyStack()
    {
        Top = null;
    }

    3 references
    public bool IsEmpty()
    {
        return Top == null;
    }
```

Linked List-based Stack

```
public void Push(int value)
{
    Node node1 = new Node();
    node1.Data = value;
    node1.Next = Top;
    Top = node1;
}
```

```
public int Pop()
{
    if (IsEmpty())
    {
        return -1;
    }
    else
    {
        int value = Top.Data;
        Top = Top.Next;
        return value;
    }
}
```

Linked List-based Stack

```
public int Count()
{
    int countr = 0;
    Node ptr = Top;

    while (ptr != null)
    {
        countr++;
        ptr = ptr.Next;
    }

    return countr;
}
```

```
public bool Find(int value)
{
    Node ptr = Top;

    while (ptr != null)
    {
        if (ptr.Data == value)
        {
            return true;
        }

        ptr = ptr.Next;
    }

    return false;
}
```

Linked List-based Stack

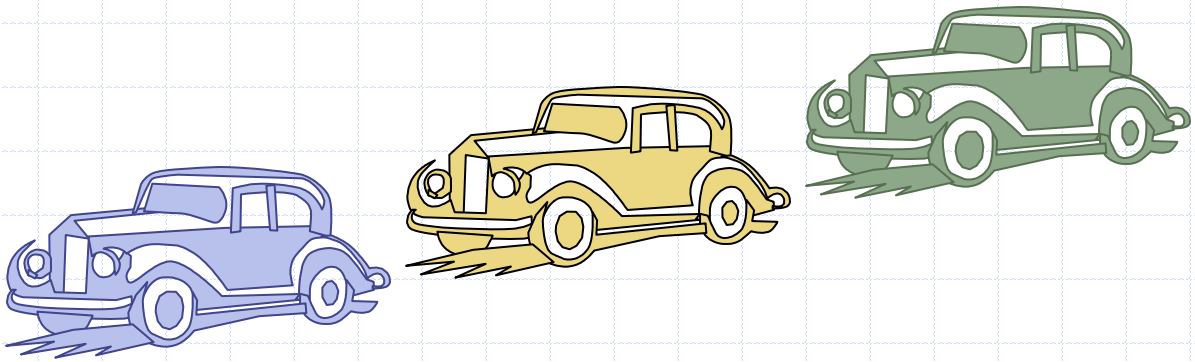
```
public void Display()
{
    if (IsEmpty())
    {
        Console.WriteLine("The stack is empty");
    }
    else
    {
        Node ptr = Top;
        Console.WriteLine("Displaying the elements of this stack");

        while (ptr != null)
        {
            Console.Write(ptr.Data + "\t");
            ptr = ptr.Next;
        }

        Console.WriteLine();
    }
}
```

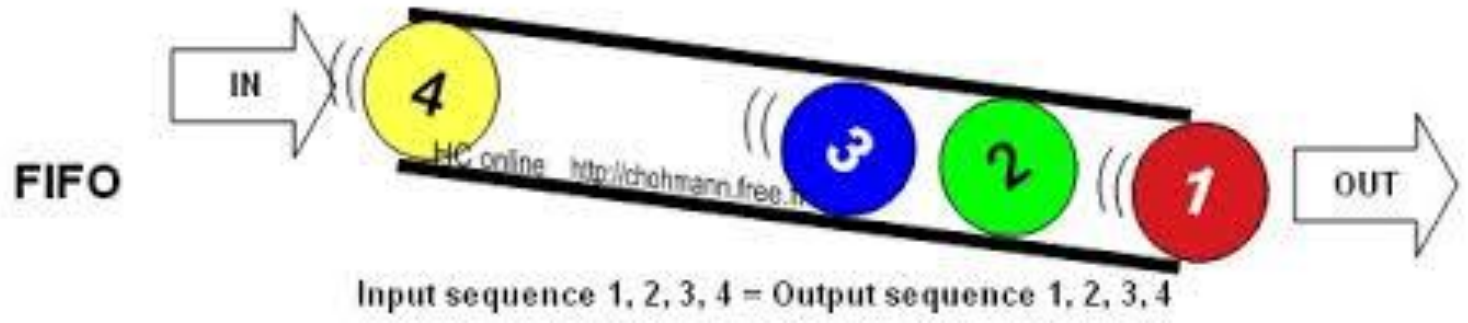
Questions?

Queues



Overview

- First-In-First-Out Data Structure



The Queue ADT

- The Queue ADT stores arbitrary objects
- Insertions and deletions follow the first-in first-out scheme
- Insertions are at the rear of the queue and removals are at the front of the queue
- **Main queue operations**:
  - enqueue(object): inserts an element at the end of the queue
  - dequeue(): removes the element at the front of the queue
- **Auxiliary queue operations**:
  - object front(): returns the element at the front without removing it
  - integer size(): returns the number of elements stored
  - boolean empty(): indicates whether no elements are stored
- Exceptions
  - Attempting the execution of dequeue or front on an empty queue throws an QueueEmpty

Queue Interface

- interface corresponding to our Queue ADT
- Requires the definition of exception QueueEmpty
- Often dequeue returns an object

```
template <typename E>
class Queue {
public:
   int size() const;
   bool empty() const;
   const E& front() const
      throw(QueueEmpty);
   void enqueue (const E& e);
   void dequeue()
      throw(QueueEmpty);
};
```

Example

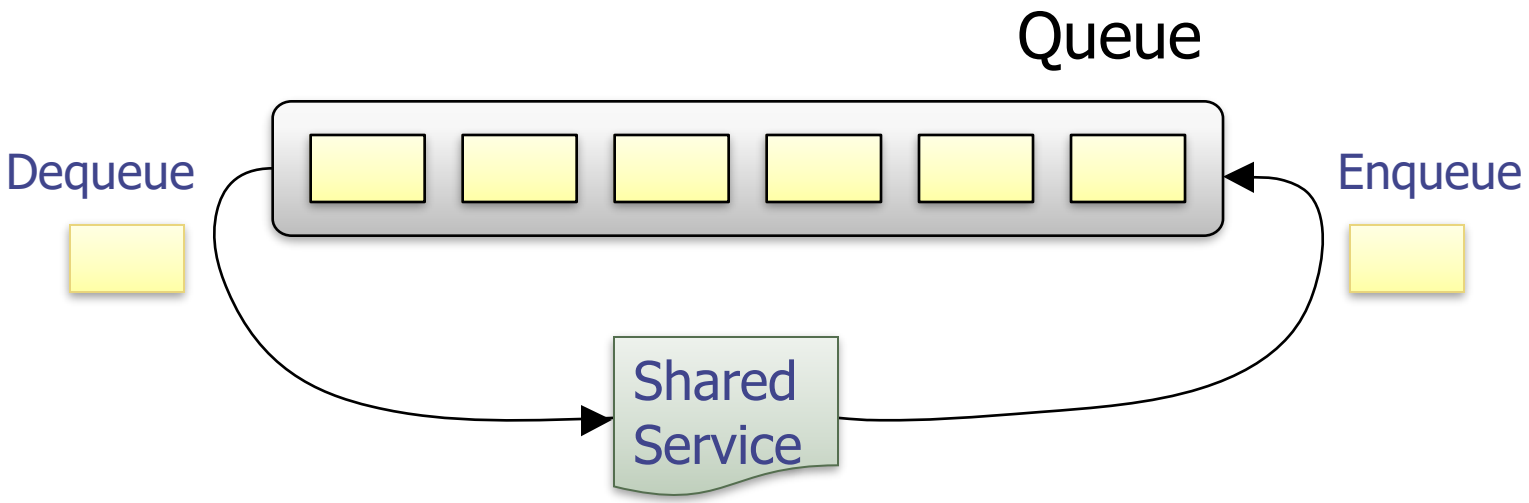
Operation	***Output***	*Q*
enqueue(5)	–	(5)
enqueue(3)	–	(5, 3)
dequeue()	–	(3)
enqueue(7)	–	(3, 7)
dequeue()	–	(7)
front()	*7*	(7)
dequeue()	–	()
dequeue()	*"error"*	()
empty()	*true*	()
enqueue(9)	–	(9)
enqueue(7)	–	(9, 7)
size()	*2*	(9, 7)
enqueue(3)	–	(9, 7, 3)
enqueue(5)	–	(9, 7, 3, 5)
dequeue()	–	(7, 3, 5)

Applications of Queues

- Direct applications
  - Waiting lists, bureaucracy
  - Access to shared resources (e.g., printer)
  - Multiprogramming
- Indirect applications
  - Auxiliary data structure for algorithms
  - Component of other data structures

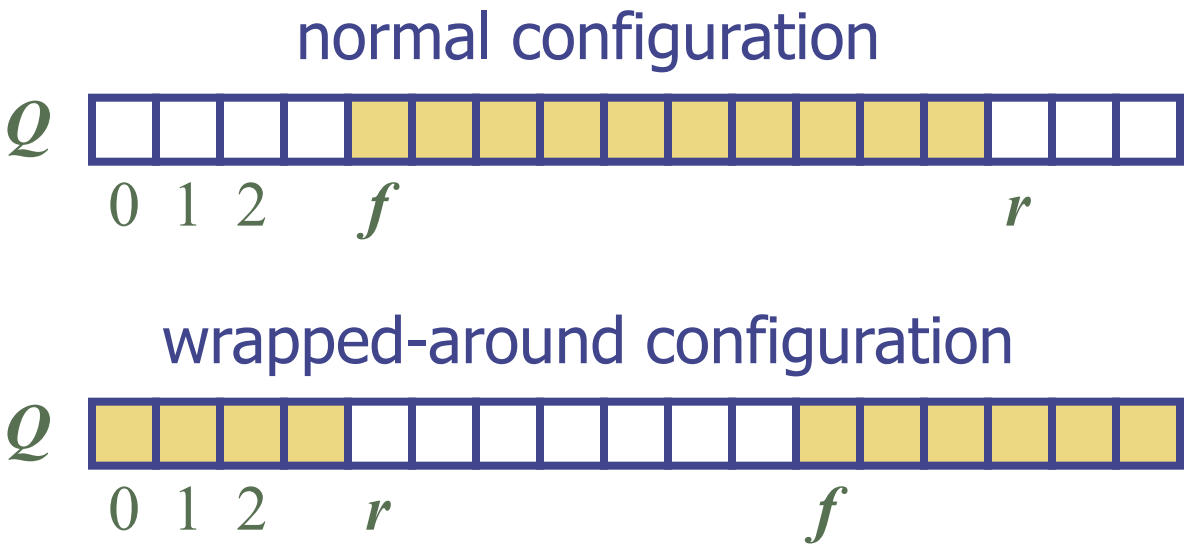
Application: Round Robin Schedulers

- We can implement a round robin scheduler using a queue Q by repeatedly performing the following steps:
  1. e = Q.front(); Q.dequeue()
  2. Service element e
  3. Q.enqueue(e)



Array-based Queue

- Use an array of size N in a circular fashion
- Three variables keep track of the front and rear
  - f index of the front element
  - r index immediately past the rear element
  - n number of items in the queue

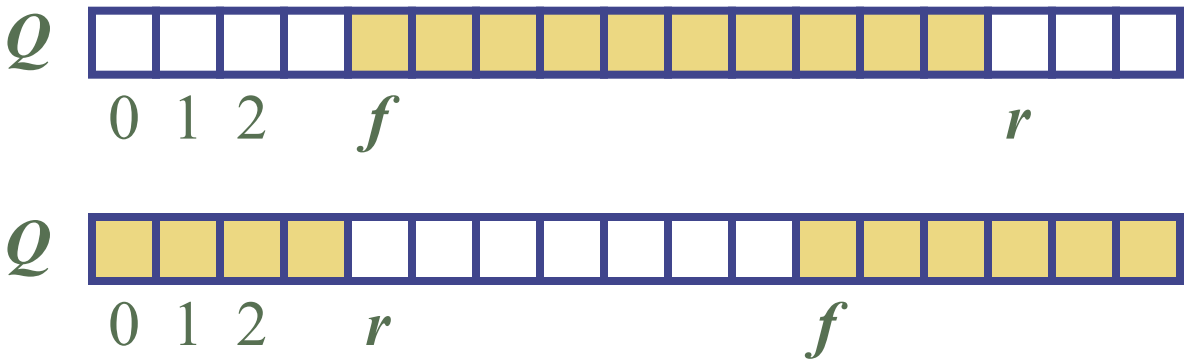


Queue Operations

- Use *n* to determine size and emptiness

```
Algorithm size()
    return n

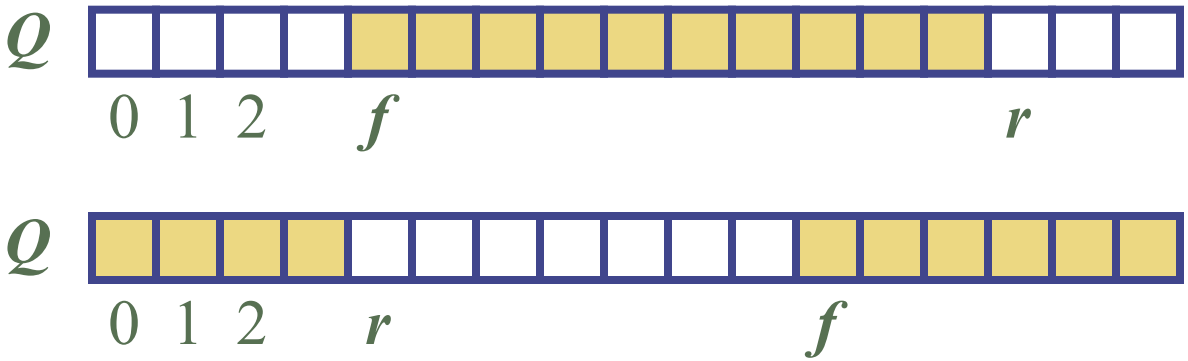
Algorithm empty()
    return (n = 0)
```



Queue Operations (cont.)

- Operation enqueue throws an exception if the array is full
- This exception is implementation-dependent

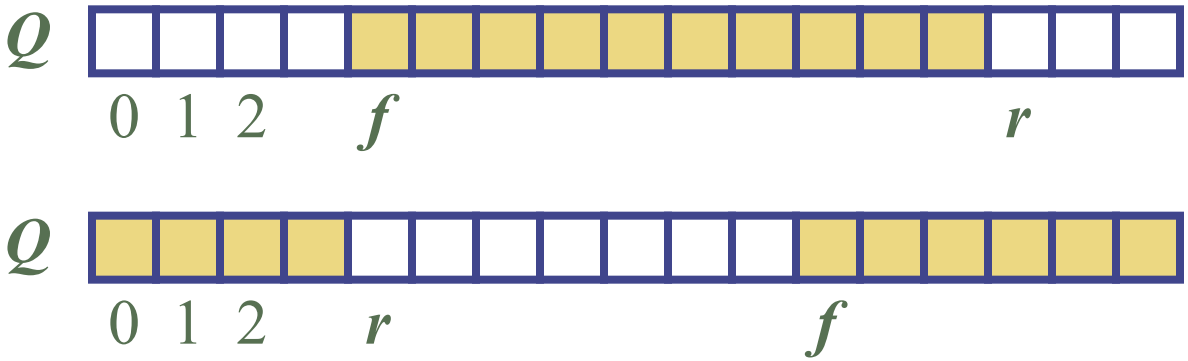
Algorithm *enqueue*(*o*)
  if *size*() = *N* – 1 **then**
    throw *QueueFull*
  else
    Q[*r*] ← *o*
    r ← (*r* + 1) mod *N*
    n ← *n* + 1



Queue Operations (cont.)

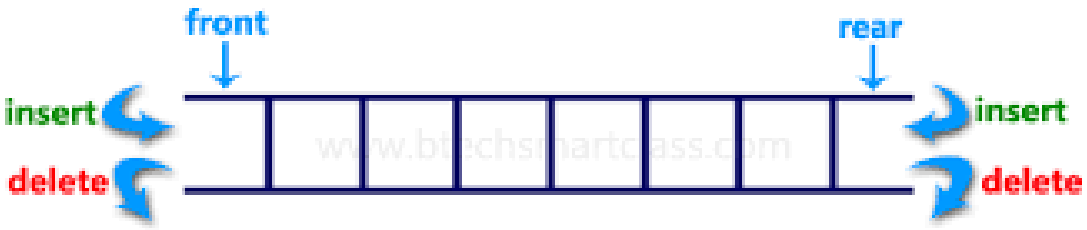
- Operation dequeue throws an exception if the queue is empty
- This exception is specified in the queue ADT

Algorithm ***dequeue***()
  if *empty*() **then**
    throw *QueueEmpty*
  else
    $f \leftarrow (f + 1) \bmod N$
    $n \leftarrow n - 1$



Double-Ended Queues (§5.3)

- Pronounce “deck”



Operation	*Output*	*D*
insertFront(3)	–	(3)
insertFront(5)	–	(5,3)
front()	*5*	(5,3)
eraseFront()	–	(3)
insertBack(7)	–	(3,7)
back()	*7*	(3,7)
eraseFront()	–	(7)
eraseBack()	–	()

DEQUE

size():	Return the number of elements in the deque.
empty():	Return true if the deque is empty and false otherwise.
push_front(e):	Insert e at the beginning the deque.
push_back(e):	Insert e at the end of the deque.
pop_front():	Remove the first element of the deque.
pop_back():	Remove the last element of the deque.
front():	Return a reference to the deque's first element.
back():	Return a reference to the deque's last element.

How to implement DEQUE?

- Question
  - Which (elementary) data structure are you going to use to implement DEQUE?
    - Array, singly linked list, doubly linked list, circular linked list
  - What happens if you use others?
- Deque by a doubly linked list

Operation	*Time*
size	$O(1)$
empty	$O(1)$
front, back	$O(1)$
insertFront, insertBack	$O(1)$
eraseFront, eraseBack	$O(1)$

Linked List-based Queue

```
class Node
{
    6 references
    public int Data;
    7 references
    public Node Next;

    1 reference
    public Node()
    {
        Next = null;
    }
}
```

```
class MyQueue
{
    10 references
    public Node Front;
    6 references
    public Node Back;
    1 reference
    public MyQueue()
    {
        Front = null;
        Back = null;
    }

    9 references
    public bool IsEmpty()
    {
        return Front == null;
    }
```

Linked List-based Queue

```
public void EnQueue(int value)
{
    Node node1 = new Node();
    node1.Data = value;
    node1.Next = null;

    if (IsEmpty())
    {
        Front = Back = node1;
    }
    else
    {
        Back.Next = node1;
        Back = node1;
    }
}
```

Linked List-based Queue

```
public int DeQueue()
{
    if (IsEmpty())
    {
        return -1;
    }
    else
    {
        int value = Front.Data;
        Front = Front.Next;
        // If the queue becomes empty, reset Back.
        if (IsEmpty())
        {
            Back = null;
        }

        return value;
    }
}
```

Linked List-based Queue

```
public int GetFront()
{
    if (IsEmpty())
    {
        throw new InvalidOperationException(
            "The queue is empty.");
    }

    return Front.Data;
}
```

```
1 reference
public int GetBack()
{
    if (IsEmpty())
    {
        throw new InvalidOperationException(
            "The queue is empty.");
    }

    return Back.Data;
}
```

```
public void Clear()
{
    Console.WriteLine(
        "Clearing the elements from the queue");

    while (!IsEmpty())
    {
        DeQueue();
    }
}
```

Linked List-based Queue

```
public int Count()
{
    int countr = 0;
    Node ptr = Front;

    while (ptr != null)
    {
        countr++;
        ptr = ptr.Next;
    }

    return countr;
}
```

```
public bool Find(int value)
{
    Node ptr = Front;

    while (ptr != null)
    {
        if (ptr.Data == value)
        {
            return true;
        }

        ptr = ptr.Next;
    }

    return false;
}
```

Linked List-based Queue

```
public void Display()
{
    if (IsEmpty())
    {
        Console.WriteLine("The queue is empty");
    }
    else
    {
        Node ptr = Front;

        Console.WriteLine(
            "Displaying the elements of this queue");

        while (ptr != null)
        {
            Console.Write(ptr.Data + "\t");
            ptr = ptr.Next;
        }

        Console.WriteLine();
    }
}
```

Questions?